

CricMetrics Pro: Complete Machine Learning Models Guide

Easy-to-Understand Guide to Every Model Used in This Project (What, When, Why, How)

About this Guide: This document explains all the Machine Learning (ML) models used in the **CricMetrics Pro** cricket player performance system. It is written in simple, clear, and easy-to-understand English so that anyone—students, teachers, coaches, and team managers—can easily understand how each model works.

1. Quick Summary of All Models

In this project, we analyze **619 real IPL cricketers** across **17 seasons (2008–2024)** using **260,920 deliveries**. We use **11 Machine Learning models** plus **2 data preparation tools** across 4 main areas:

Area	Goal	Models Used	Best Model
1. Regression	Predict a continuous player rating (50 to 95) and fair auction price (₹ Crores)	- Linear Regression - Polynomial Regression - Random Forest Regressor - Neural Network (MLP)	Random Forest Regressor ($R^2 = 0.9789$, Error = 1.25 pts)
2. Classification	Put players into 3 talent tiers: - Elite / Marquee (Tier 1) - Core / Star (Tier 2) - Developing / Squad (Tier 0)	- K-Nearest Neighbors (KNN) - Random Forest Classifier - Neural Network (MLP) - Logistic Regression - Decision Tree	K-Nearest Neighbors (KNN) (95.16% Accuracy) & Random Forest (94.35%)
3. Clustering	Group players by tactical playing style without using preset labels	K-Means Clustering ($k=5$)	5 Tactical Archetypes (Anchor, Finisher, Fast Bowler, Spinner, All-Rounder)
4. Dimensionality Reduction	Compress 29 complex statistics into an easy 2D map (X, Y)	Principal Component Analysis (PCA)	2D Latent Map (Explains 60.8% of all variation)
5. Data Preparation	Clean and scale data so models can read it properly	- StandardScaler (Z-score scaling) - One-Hot Encoding	StandardScaler ($\mu=0, \sigma=1$)

2. Data Preparation Tools

Before feeding cricket numbers into machine learning models, we must prepare the data so computers can read it fairly.

2.1 StandardScaler (Z-Score Normalization)

- WHAT is it?

A tool that rescales all numbers so that their average is 0 and standard deviation is 1. It turns big numbers and small numbers into the same standard scale:

$$\text{Scaled Value } z = \frac{\text{Value} - \text{Average}}{\text{Spread (Std Dev)}}$$

- **WHEN is it used?**

Immediately after splitting data into training (80%) and testing (20%). It is fitted only on training data and then used to scale test data and live user inputs in the web app.

- **WHY do we need it?**

In cricket, `total_runs` can be huge (like 8,000 runs for Virat Kohli), while `economy_rate` is small (like 7.2 runs per over). If we don't scale them, distance-based models like KNN, K-Means, and PCA will pay 99% attention to runs and completely ignore bowling economy. Scaling makes every stat count fairly.

- **HOW is it implemented?**

We use `StandardScaler()` from Scikit-Learn. Saved to disk as `models/scaler.joblib`.

2.2 One-Hot Encoding

- **WHAT is it?**

A method that converts text categories (like `"All-Rounder"` or `"Specialist Bowler"`) into simple 0 and 1 columns that computers can calculate.

- **WHEN is it used?**

During the initial data cleaning step in `src/cricket_data_pipeline.py`.

- **WHY do we need it?**

Computers only understand numbers. But if we labeled roles as `1 = Batter`, `2 = All-Rounder`, `3 = Bowler`, the model would wrongly assume that a Bowler is "3 times bigger" than a Batter. One-hot encoding creates separate True/False (1/0) switches so no artificial rank is forced.

- **HOW is it implemented?**

Using Pandas `pd.get_dummies(df, columns=['primary_role'], drop_first=True)`. We drop the first column to avoid redundant mathematical loops (called the dummy variable trap).

3. Supervised Learning: Regression Models (Predicting Player Rating)

Regression models predict a continuous number (here, the player's overall rating between 50 and 95).

3.1 Linear Regression (Ordinary Least Squares – OLS)

- **WHAT is it?**

The most basic and well-known regression model. It draws a straight line (or flat plane) through the data. It gives each stat a fixed weight (coefficient):

$$\text{Rating} = \beta_0 + (\beta_1 \times \text{Runs}) + (\beta_2 \times \text{Strike Rate}) + (\beta_3 \times \text{Wickets}) + \dots$$

- **WHEN is it used?**

Used in Tab 3 and Tab 6 as our baseline reference model.

- **WHY do we need it?**

It is simple, extremely fast, and easy to interpret. You can look at the weights and clearly see: "Every extra Player of the Match award adds +0.8 points to the rating."

- **HOW does it work & performance?**

- It finds the line that minimizes the sum of squared differences between real ratings and predicted ratings.

- **Test R^2 Score:** `0.9490` (explains 94.9% of rating variance).

- **Average Error (RMSE):** 1.95 points out of 100.
 - **Weakness:** It assumes performance grows in a straight line, but cricket has sharp jumps that straight lines cannot capture.
-

3.2 Polynomial Regression (Degree 2)

- **WHAT is it?**

An upgrade to Linear Regression that also looks at combinations and squares of stats (for example: $\text{Strike Rate} \times \text{Boundary \%}$ or $\text{Death Economy} \times \text{Dot Ball \%}$).

- **WHEN is it used?**

Used when testing whether stats multiply each other's value.

- **WHY do we need it?**

Cricket skills are multiplicative! A strike rate of 160 is good, but having a strike rate of 160 **together with** a high boundary percentage (>65%) is massively more dangerous in death overs than hitting 160 with risky singles. Polynomial regression tests this synergy.

- **HOW does it work & performance?**

- We use Scikit-Learn's `PolynomialFeatures(degree=2)` on key impact stats, then fit a `LinearRegression()` model.
 - **Test R^2 Score:** 0.9087 .
 - **Average Error (RMSE):** 2.61 points.
 - **Weakness:** Creating pairs of features increases the number of columns, which causes slight overfitting compared to tree models.
-

3.3 Random Forest Regressor (Ensemble of Decision Trees) — 🏆 WINNER

- **WHAT is it?**

A team (ensemble) of **180 decision trees**. Each tree looks at random subsets of players and stats, makes its own prediction, and the forest takes the average of all 180 trees as the final answer.

- **WHEN is it used?**

This is the main production engine running in Tab 1, Tab 3, and Tab 5 of the Streamlit app. It powers the live auction purse calculator.

- **WHY do we need it?**

1. **Best Accuracy in the Project:** Achieved an outstanding $R^2 = 0.9789$ and lowest error (RMSE = 1.25 points).
2. **Handles Sudden Cricket Thresholds:** In T20 cricket, giving away 6.5 runs per over in the death overs (overs 16–20) is game-winning, while giving away 11.5 is disastrous. Decision trees effortlessly split players at these exact thresholds (e.g., `IF Death Econ < 7.5 THEN add big rating bonus`).
3. **No Overfitting:** Averaging 180 trees cancels out random noise and mistakes from individual trees.

- **HOW does it work & performance?**

- Built using `RandomForestRegressor(n_estimators=180, max_depth=12, random_state=42)` .
 - **Test R^2 Score:** 0.9789 (97.89% accurate).
 - **Average Error (RMSE):** 1.2562 points (very small error).
 - **Mean Absolute Error (MAE):** 0.9024 points.
-

3.4 Multi-Layer Perceptron (MLP) Regressor (Neural Network)

- **WHAT is it?**

An artificial neural network (deep learning model) inspired by the human brain. It has:

- Input layer: 29 stats
- Hidden layer 1: 64 neurons (processing nodes)
- Hidden layer 2: 32 neurons
- Output layer: 1 continuous rating value
- **WHEN is it used?**

Used as a benchmark to test if deep learning is better than decision trees on tabular cricket data.

- **WHY do we need it?**

Fulfills **Module IX** (Perceptrons & Neural Networks) of the Machine Learning syllabus. It tests whether complex multi-layer connections can discover hidden player patterns.

- **HOW does it work & performance?**

- Data passes forward through weighted connections. When it makes an error, it uses backpropagation (Adam optimizer) to adjust its weights.
- **Test R^2 Score:** 0.5495 .
- **Average Error (RMSE):** 5.80 points.
- **Key Lesson:** Confirms the standard rule in data science: **Tree models (like Random Forest) perform much better on spreadsheet/tabular data with ~600 rows than deep neural networks**, because neural networks need tens of thousands of rows to train properly without getting confused.

4. Supervised Learning: Classification Models (Talent Tiers)

Classification models assign a player to one of **3 discrete talent categories**:

- **Tier 1 (Elite / Marquee):** Rating ≥ 80.0 (Match winners, captaincy candidates, high-value auction picks).
- **Tier 2 (Core / Star):** Rating 68.0 to 79.9 (Consistent tournament starters and reliable anchors).
- **Tier 0 (Developing / Squad):** Rating < 68.0 (Emerging domestic prospects and squad backup options).

4.1 K-Nearest Neighbors (KNN, $k=5$) — 🏆 TOP ACCURACY

- **WHAT is it?**

A model that classifies a player by finding the **5 most similar players** in cricket history and choosing the most common tier among those 5 neighbors.

- **WHEN is it used?**

Used in Tab 2 and Tab 6 for talent tier classification.

- **WHY do we need it?**

1. **Highest Accuracy:** Achieved **95.16% test accuracy** (top among all classifiers).
2. **Matches Real Cricket Scouting:** When a franchise scout looks at an uncapped player, they naturally ask: "*Who does this player resemble?*" If a young player matches Hardik Pandya, Ben Stokes, and Andre Russell in batting impact, strike rate, and overs bowled, they naturally belong in Tier 1.

- **HOW does it work & performance?**

- Calculates Euclidean distance across all 29 scaled stats:
$$\text{Distance} = \sqrt{\sum (x_i - y_i)^2}$$
- Takes the 5 closest players ($k=5$) and takes a majority vote.
- **Test Accuracy:** 95.16% .

- **Macro F1-Score:** 0.9501 (very balanced across all 3 tiers).
-

4.2 Random Forest Classifier — 🏆 PRODUCTION TIER ENGINE

- **WHAT is it?**

An ensemble of **180 decision trees** voting on which tier a player belongs to.

- **WHEN is it used?**

Powers the Tier Classification engine and the **Feature Importance Leaderboard** in Tab 6.

- **WHY do we need it?**

1. **Very High Accuracy:** 94.35% on test data, and highest cross-validation score (94.75%).

2. **Explainability (Gini Importance):** Unlike "black-box" models, Random Forest tells team owners **why** it made a decision by ranking the top stats that decided the tiers.

- **HOW does it work & performance?**

- Built using `RandomForestClassifier(n_estimators=180, criterion='gini', random_state=42)`.

- **Test Accuracy:** 94.35% .

- **5-Fold CV Accuracy:** 94.75% .

- **Top Factors Identified:** Clutch Match-Winner Index (12.8%), Batting Impact Index (11.6%), Player of the Match awards (7.8%).
-

4.3 Decision Tree Classifier (CART)

- **WHAT is it?**

A visual flowchart that asks simple True/False questions step-by-step:

- *Question 1:* Is `clutch_match_winner_index` > 24.5?
- *If Yes:* Is `batting_impact_index` > 62.0? $\rightarrow$ **Elite Tier**
- *If No:* Is `wickets_taken` < 8? $\rightarrow$ **Developing Tier**

- **WHEN is it used?**

Used when coaches or non-technical executives want a clear, rule-by-rule explanation without math formulas.

- **WHY do we need it?**

It is the most interpretable model in machine learning. Anyone can trace the decision path on a sheet of paper.

- **HOW does it work & performance?**

- Uses the Gini Impurity formula to pick questions that cleanest separate the groups. Max depth is limited to 6 to prevent memorizing the data.
 - **Test Accuracy:** 87.90% .
 - **Macro F1-Score:** 0.8812 .
-

4.4 Logistic Regression (Multinomial / Softmax)

- **WHAT is it?**

A linear classification model that calculates the probability (from 0% to 100%) that a player belongs to each tier:

$$P(\text{Elite}) = 82\%, \quad P(\text{Core}) = 15\%, \quad P(\text{Developing}) = 3\%$$

- **WHEN is it used?**

Used as the standard baseline probabilistic classifier.

- **WHY do we need it?**

Franchise directors like to see risk percentages during auction bidding wars rather than just a flat "Yes/No" label.

- **HOW does it work & performance?**
 - Computes linear scores for each tier and uses the Softmax mathematical function to turn scores into probabilities that sum to 100%.
 - **Test Accuracy:** 87.90% .
 - **Macro F1-Score:** 0.8853 .
-

4.5 Multi-Layer Perceptron (MLP) Classifier (Neural Network)

- **WHAT is it?**

A neural network classifier with 2 hidden layers (64 neurons and 32 neurons) and a Softmax output layer with 3 units.
 - **WHEN is it used?**

Benchmark multi-class neural network classifier (Module IX).
 - **WHY do we need it?**

Tests if connected non-linear neurons can draw flexible boundaries separating borderline players.
 - **HOW does it work & performance?**
 - Trained using Cross-Entropy loss and Adam optimizer.
 - **Test Accuracy:** 92.74% .
 - **Macro F1-Score:** 0.9287 .
 - **Finding:** Performed well (92.7%), but KNN and Random Forest were faster, more accurate, and easier to explain to coaches.
-

5. Unsupervised Learning: Clustering (Tactical Archetypes)

Unlike regression and classification, clustering has **no correct answer labels**. The computer looks at all players and groups them by similar playing styles on its own.

5.1 K-Means Clustering ($k=5$)

- **WHAT is it?**

An algorithm that groups 619 cricketers into **5 tactical clusters** based on how close their stats are in multi-dimensional space.
- **WHEN is it used?**

Powers **Tab 4 (Tactical Archetypes & Roster Balance)** in the web app.
- **WHY do we need it?**

Nominal labels like "Batter" or "Bowler" are too simple for modern T20 cricket:

 - Virat Kohli and Andre Russell are both listed as "Batters", but they play completely different roles (Kohli anchors the innings; Russell hits explosive boundaries in death overs).
 - Jasprit Bumrah and Yuzvendra Chahal are both "Bowlers", but Bumrah bowls 145 km/h yorkers at the death while Chahal spins middle-over webs.
- K-Means automatically discovered these **5 real tactical archetypes**:
 1. **Cluster 0: Tactical Anchor & Top-Order Accumulator** (builds partnerships, high average).
 2. **Cluster 1: High-Impact Pace Spearhead & Death Bowler** (yorkers, low death economy, high wickets).
 3. **Cluster 2: Explosive Death-Over Finisher & Boundary Hitter** (strike rate > 155, high sixes).

4. **Cluster 3: Mystery / Control Spin Maestro** (high dot ball %, economical middle overs).

5. **Cluster 4: Elite Dual-Threat All-Rounder** (contributes heavily with both bat and ball).

- **HOW does it work?**

1. We tested cluster counts from $k=2$ to $k=7$ using the **Elbow Curve** (measuring cluster tightness/inertia) and **Silhouette Score** (measuring how distinct clusters are from each other).

2. $k=5$ gave the clearest real-world cricket separation ($s = 0.285$).

3. Players are assigned to the nearest cluster center (centroid).

- Saved as `models/kmeans_model.joblib`.

6. Dimensionality Reduction (Visualizing High-Dimensional Data)

6.1 Principal Component Analysis (PCA, 2 Components)

- **WHAT is it?**

A mathematical technique that compresses 29 different statistics down into **2 summary numbers (Component 1 and Component 2)** so that all 619 players can be drawn on a standard 2D scatter plot (X and Y axis).

- **WHEN is it used?**

Powers the interactive 2D Tactical Scatter Plot in **Tab 4** of the web app.

- **WHY do we need it?**

Human eyes and computer screens can only view 2 or 3 dimensions at once. Looking at a 29-column spreadsheet is overwhelming. PCA keeps the most important information while allowing coaches to see every player's tactical position on a single screen.

- **HOW does it work & performance?**

- It finds the two directions where player statistics vary the most (using eigenvalues of the covariance matrix):

- **Principal Component 1 (X-axis, 38.16% variance):** Represents general match volume and career impact (runs, wickets, matches).

- **Principal Component 2 (Y-axis, 22.63% variance):** Represents bowling vs. batting specialization (bowlers go up, batters go down, all-rounders stay in the middle).

- **Together, these 2 numbers capture 60.79% of all information** in the 29 original columns.

- Saved as `models/pca_model.joblib`.

7. Model Evaluation & Validation Techniques

To ensure our models actually work on unseen matches and don't just memorize the past, we use rigorous testing methods:

1. **80/20 Stratified Train/Test Split (Module VI):**

- 495 players (80%) are used to train the models.
- 124 players (20%) are locked away in a vault and only used to test final performance.
- "Stratified" means both sets have the exact same percentage of Elite, Core, and Developing players.

2. **5-Fold Cross-Validation (Module VI):**

- The training set is split into 5 equal parts. The model trains on 4 parts and tests on the 5th part, repeating this 5 times.

- We report the average and standard deviation (e.g. 0.9528 ± 0.0129). This proves the model is reliable and didn't just get lucky.

3. Evaluation Metrics Used:

- **R^2 Score (Coefficient of Determination):** How much rating variation is explained (1.0 is perfect; Random Forest got 0.9789).
- **RMSE (Root Mean Squared Error):** Average error in rating points (1.25 points).
- **Accuracy:** Percentage of correct tier predictions (KNN got 95.16%).
- **Macro F1-Score:** Balances precision and recall across small and large tiers so minority classes aren't ignored.
- **Confusion Matrix:** A 3×3 grid showing where the model was right and where it made mistakes.

8. Master Model Leaderboard & Comparison

Model Name	Task	Syllabus Module	Test Score	5-Fold CV Score	Why We Chose It
Random Forest Regressor	Continuous Rating	Module VIII	$R^2 = 0.9789$ (RMSE: 1.25)	0.9528 ± 0.0129	Production Winner: Highest accuracy, handles non-linear cricket thresholds.
Linear Regression (OLS)	Continuous Rating	Module IV	$R^2 = 0.9490$ (RMSE: 1.95)	0.8797 ± 0.0371	Baseline model; easy to explain exact weights.
Polynomial Regression	Continuous Rating	Module IV	$R^2 = 0.9087$ (RMSE: 2.61)	0.8739 ± 0.0343	Tests multiplicative synergy between strike rate and boundaries.
MLP Regressor (Neural Net)	Continuous Rating	Module IX	$R^2 = 0.5495$ (RMSE: 5.81)	0.4503 ± 0.1375	Neural network baseline; shows trees are better for tabular sports data.
K-Nearest Neighbors (KNN)	Tier Classification	Module V	95.16% Acc (F1: 0.9501)	0.9030 ± 0.0221	Accuracy Winner: Classifies players by finding historical peer matches.
Random Forest Classifier	Tier Classification	Module VIII	94.35% Acc (F1: 0.9459)	0.9475 ± 0.0142	Production Winner: Stable across folds and gives Top 10 feature rankings.
MLP Classifier (Neural Net)	Tier Classification	Module IX	92.74% Acc (F1: 0.9287)	0.8141 ± 0.0384	Deep learning classifier; draws non-linear tier boundaries.
Logistic Regression	Tier Classification	Module V	87.90% Acc (F1: 0.8853)	0.9253 ± 0.0180	Linear probabilistic model; gives exact risk percentages.
Decision Tree (CART)	Tier Classification	Module V	87.90% Acc (F1: 0.8812)	0.8990 ± 0.0195	100% human-readable "IF-THEN" flowchart for coaches.
K-Means Clustering	Tactical Styles	Module VII	5 Clusters ($s = 0.285$)	Validated via Elbow Curve	Discovered 5 real playing styles beyond simplistic Batter/Bowler tags.
PCA	2D Visualization	Module VIII	60.79% Var (2 Components)	SVD Decomposition	Compresses 29 stats into an interactive 2D scatter plot.

Key Takeaway for Viva & Interviews

When asked: "Why did you use these specific models?"

Answer:

"Every model was chosen to fulfill our university syllabus and solve a real sports business problem. For ratings, **Random Forest Regressor** won ($R^2 = 0.9789$) because cricket performance has sharp non-linear thresholds in death overs that straight lines cannot capture. For talent tiers, **K-Nearest Neighbors** achieved top test accuracy (95.16\%) because evaluating players against similar historical peers is the natural way sports scouting works. Finally, **K-Means (k=5)** and **PCA** allow coaches to discover tactical playing styles and visualize complex 29-dimensional performance on a simple 2D map."